CAHIER DES CHARGES

Studio d’automatisation YouTube assisté par IA

Production de vidéos originales sur la science, les inventions, les innovations, les technologies et les grandes questions. Vidéos longues + Shorts, voix off IA, visuels 2D/3D, montage automatisé et pilotage par Telegram.

Version	1.0 — spécification fonctionnelle et technique
Date	16 octobre 2026
Budget cible	0 € de frais récurrents au démarrage, sous réserve des quotas gratuits
Principe	Automatiser la production, mais garder une validation humaine avant publication

Décision structurante : architecture « provider-agnostic ». Gemini TTS, les modèles de langage, les moteurs de rendu 2D/3D et YouTube doivent être des fournisseurs interchangeables, sans dépendance unique dans le cœur du système.

Le document guide le développement, les tests et le déploiement. Tarifs, quotas API et règles des plateformes doivent être revérifiés avant la mise en production.

Sommaire

1. Vision et objectifs

2. Périmètre fonctionnel

3. Éditorial, RPM et monétisation

4. Stack et arbitrages

5. Architecture et états

6. Workflow complet

7. Exigences détaillées

8. Bot Telegram : commandes

9. Données et structure du dépôt

10. Qualité, sécurité et observabilité

11. Tests et critères d’acceptation

12. Roadmap de réalisation

13. Déploiement et configuration

14. Risques et décisions

15. Sources officielles

Résumé exécutif

Construire une application qui transforme une idée en dossier éditorial, script sourcé, storyboard, voix off, visuels originaux, montage, sous-titres, variantes 16:9 et 9:16, métadonnées et rapport de contrôle. Telegram sert de télécommande : création, suivi, validation, liste des vidéos, rappels, calendrier et statistiques. La publication publique est désactivée par défaut et nécessite une approbation explicite.

Aucune stack ne peut garantir un RPM précis ou une monétisation durable. Le système doit maximiser l’originalité, la fiabilité, les droits clairs, la valeur pédagogique et la variation réelle entre vidéos.

1. Vision et objectifs

1.1 Vision produit

Créer un studio éditorial sans visage pour une ou plusieurs chaînes YouTube. Les sujets peuvent couvrir la science, les inventions, les technologies, l'innovation, les objets du quotidien, les expériences expliquées, les paradoxes et les questions « et si… ? », en séparant clairement les faits, les hypothèses et les simulations.

- Automatiser les tâches répétitives : recherche préparatoire, scripts, narration, sous-titres, rendu, exports et rapports.
- Créer une identité visuelle reconnaissable sans produire des vidéos interchangeables.
- Produire des vidéos longues et des Shorts autonomes ou dérivés, en français et éventuellement en anglais.
- Piloter les tâches depuis Telegram sans exposer les clés API ni autoriser les utilisateurs non autorisés.
- Conserver une piste d'audit des sources, versions, prompts, licences, validations et publications.

1.2 Indicateurs initiaux

Indicateur	Cible initiale	Note
Validation humaine	100 % avant publication	Au MVP, pas d'upload public silencieux.
Traçabilité	Chaque vidéo a sources + manifeste d'assets	Permet audit et corrections.
Formats	16:9 + 9:16	Le vertical peut avoir son propre storyboard.
Coût logiciel	0 € cible	Sous réserve des quotas et conditions des offres gratuites.
Qualité	Aucun export sans QA réussi	Les contrôles automatiques ne remplacent pas la revue humaine.

1.3 Hors périmètre du MVP

- Publier en masse sans revue éditoriale ou chercher à contourner les règles de YouTube.
- Cloner la voix d'une personne sans consentement.
- Utiliser automatiquement des médias dont les droits sont inconnus.
- Promettre un RPM ou une admission au Programme Partenaire YouTube.
- Rendre obligatoire un logiciel payant comme After Effects ou un seul fournisseur d'IA.

2. Périmètre fonctionnel

Module	Responsabilités	Priorité
Idées & tendances	Proposer des sujets, noter l'intérêt, détecter les doublons.	MVP
Recherche & sources	Collecter les sources et créer un registre des affirmations vérifiables.	MVP
Script éditorial	Hooks, plan, narration, exemples, limites, conclusion et CTA.	MVP
Vérification factuelle	Signaler claims sans source, chiffres et citations à vérifier.	MVP
Storyboard	Découper en scènes avec durée, narration, visuel, texte et mouvement.	MVP
Visuels	SVG, graphiques, Manim, Blender, Three.js ou images autorisées.	MVP
Voix off	TTS par segments, langue, voix, rythme, contrôle de durée.	MVP
Montage & sous-titres	FFmpeg, SRT/VTT, mixage, exports horizontal et vertical.	MVP
Packaging	Titres, description, chapitres, miniature et note de divulgation IA.	MVP
Telegram	Commandes, notifications, approbations, calendrier et rapports.	MVP
Publication	Upload privé puis programmation après validation.	V1
Analytics	Suivre métriques et comparer les résultats par thème/format.	V1
Amélioration	Tests éditoriaux basés sur les métriques, sans sacrifier la qualité.	V2

Chaque module doit être isolé : une panne TTS ne doit pas effacer le script ou imposer de refaire les recherches.

3. Éditorial, RPM et monétisation

3.1 Pistes éditoriales compatibles

- Science accessible : phénomènes physiques, espace, matière, cerveau, énergie.
- Inventions et innovations : comment elles fonctionnent, leur histoire, limites et conséquences.
- Technologies : IA, robotique, batteries, semi-conducteurs, informatique, réseaux.
- Objets et infrastructures du quotidien : mécanismes, matériaux et systèmes invisibles.
- Questionnements et hypothèses : « et si… ? », paradoxes, scénarios clairement présentés comme hypothétiques.
- Comparaisons techniques : deux approches, avantages, compromis, coûts et limites.

3.2 RPM : ce qu'il faut comprendre

Le RPM n'est pas garanti par une niche. Il dépend notamment du pays des spectateurs, de la saison publicitaire, du sujet, du format, de la durée, du temps de visionnage, de l'inventaire publicitaire et de l'éligibilité de la vidéo. La science et la technologie peuvent intéresser des annonceurs, mais cela ne garantit pas un RPM élevé. Les vidéos longues et les Shorts doivent être analysés séparément, car leurs mécanismes de revenus diffèrent.

Le bot doit suivre les métriques réelles de chaque chaîne et langue. Ne pas choisir les sujets uniquement en fonction d'un RPM supposé : suivre aussi la rétention, la satisfaction, le CTR, le temps de visionnage et la fidélisation.

3.3 Règles pour réduire le risque de refus de monétisation

- Chaque vidéo doit apporter une valeur originale : explication, démonstration, angle, comparaison ou analyse distincte.
- Éviter les épisodes interchangeables : même narration générique, même diaporama et quelques mots changés.
- Ne pas se limiter à des images IA en diaporama avec une voix off générique ; les visuels doivent réellement expliquer le sujet.
- Ne pas recopier scripts, articles, vidéos ou publications. Écrire une narration originale et citer les sources.
- Enregistrer la provenance et les droits des images, sons, musiques, polices, textures et modèles 3D.
- Ne pas présenter une simulation comme une vraie prise de vue. Ajouter des indications comme « illustration », « simulation » ou « schéma non à l'échelle » si nécessaire.
- Imposer une revue renforcée aux sujets sensibles, controversés ou à conséquences pratiques.
- Ne pas publier massivement sans contrôle éditorial.

3.4 Divulgation IA

YouTube demande une divulgation lorsque du contenu généré ou modifié paraît réaliste et modifie matériellement la représentation d'une personne, d'un lieu ou d'un événement réel. Les animations stylisées, sous-titres et aides à la rédaction ne déclenchent pas automatiquement la même obligation. Le pipeline doit inclure un champ « divulgation requise ? », la raison et une étape de confirmation. Toute divulgation requise doit être respectée.

Il est impossible de garantir l'absence de perte de monétisation : la décision appartient à YouTube et les règles peuvent évoluer. Ce cahier des charges réduit les risques, sans promettre un résultat.

4. Stack et arbitrages

4.1 Stack de départ recommandée

Couche	Choix de départ	Pourquoi / alternatives
Orchestration	Python 3.12+ ; FastAPI si API nécessaire	Écosystème adapté au média et aux tâches asynchrones.
Jobs	RQ/Redis ou Dramatiq ; SQLite au MVP, PostgreSQL ensuite	Queue, reprise après erreur et suivi de progression.
Voix off	Gemini TTS via adaptateur ; Piper en fallback	Gemini TTS offre un contrôle de style ; vérifier les quotas et conditions actuels.
LLM / scripts	Adaptateur multi-fournisseurs	Gemini, modèles API accessibles ou modèles locaux.
Visuels 2D	SVG, Pillow, ImageMagick, graphiques	Reproductibles et éditables.
Animation	Manim pour schémas/maths ; Blender pour 3D ; Three.js selon besoin	Rendu scriptable et adaptable.
Montage	FFmpeg	Assemblage, encodage, audio, sous-titres, recadrage.
Sous-titres	Alignement sur script ou Whisper local + relecture	SRT/VTT et sous-titres incrustés optionnels.
Telegram	Python + aiogram ou python-telegram-bot	Commandes, boutons et notifications.
Publication	YouTube Data API v3 + OAuth2	Upload et programmation sous réserve des permissions et quotas.
Dashboard facultatif	Next.js + TypeScript + Tailwind	Prévisualisation, filtres et analytics plus tard.

4.2 Gemini TTS : choix réaliste

La documentation officielle de Gemini décrit une TTS contrôlable, avec choix de voix et paramètres de style, et la tarification affiche un niveau gratuit pour certains modèles. Ce n'est pas une garantie d'usage illimité : modèles, quotas, conditions de données et prix peuvent changer. Encapsuler Gemini derrière une interface TTSProvider, tester la qualité en français/anglais et surveiller la consommation. En cas de quota dépassé, basculer vers Piper ou placer le job en attente.

- Vérifier dans le compte Google utilisé le modèle, les limites, le coût exact et les conditions applicables.
- Ne jamais mettre une clé API dans Telegram, le navigateur, le dépôt Git ou les logs.
- Enregistrer modèle, voix, langue, texte exact, paramètres, durée et hash de chaque audio.
- Prévoir des tentatives limitées, des délais progressifs et un fallback local.
- Vérifier le traitement des données : l'offre gratuite peut avoir des conditions différentes de l'offre payante.

4.3 GPT-6/Astra, Claude Code et génération 3D

Un assistant de langage ou de programmation peut générer du code Blender Python, Manim, SVG, Three.js ou des scripts de montage. Il ne faut pas confondre la génération de code avec un rendu 3D final fiable : le worker exécute le code, rend une prévisualisation, capture les erreurs et soumet le résultat au contrôle visuel. After Effects reste une intégration externe facultative, pas une dépendance du MVP ; pour une stack sans abonnement, privilégier Blender, Manim, SVG et FFmpeg.

4.4 Interfaces de fournisseurs

Définir LLMProvider, TTSProvider, ImageProvider, SceneRenderer, SubtitleProvider, YouTubePublisher et NotificationProvider. Chaque adaptateur expose ses capacités, langues, formats, limites, coûts estimés et erreurs normalisées. Le code métier ne doit pas appeler directement une marque de modèle.

5. Architecture et états

5.1 Vue logique

```
Telegram Bot / dashboard facultatif ■■ API + authentification ■■ Orchestrateur de workflows ■ ■■ Recherche & sources ■ ■■ Script & contrôle factuel ■ ■■ Storyboard & gestion des assets ■ ■■ Voix off TTS ■ ■■ Sous-titres & rendu vidéo ■ ■■ Contrôle qualité ■ ■■ Packaging & miniature ■ ■■ Publication YouTube ■ ■■ Analytics & rapports ■■ File de tâches + workers isolés ■■ Base de données ■■ Stockage : sources, scripts, audio, scènes, exports, logs
```

5.2 États du pipeline

État	Signification	Transition
IDEA	Sujet capturé	RESEARCH_PENDING / REJECTED
RESEARCHED	Sources et angle disponibles	SCRIPT_DRAFT
SCRIPT_DRAFT	Script généré	FACT_CHECK / NEEDS_EDIT
FACT_CHECK	Affirmations contrôlées	SCRIPT_APPROVED / NEEDS_EDIT
STORYBOARD	Scènes définies	ASSET_GENERATION
ASSETS_READY	Visuels disponibles et droits vérifiés	VOICE_READY / RENDER_PENDING
VOICE_READY	Narration générée et vérifiée	RENDER_PENDING
RENDERING	Rendu en cours	QA_PENDING / FAILED
QA_PENDING	Contrôles automatiques terminés	HUMAN_REVIEW / NEEDS_FIX
HUMAN_REVIEW	Revue humaine en attente	APPROVED / NEEDS_FIX / REJECTED
APPROVED	Validation explicite	UPLOAD_PRIVATE / SCHEDULED
PUBLISHED	Publication confirmée	ANALYTICS_TRACKING
FAILED	Échec et erreur enregistrés	RETRY / CANCELLED

5.3 Résilience

- Chaque tâche a un ID, un numéro de tentative et un état explicite.
- Relancer un job ne doit pas dupliquer une publication ou écraser un artefact validé.
- Réutiliser les artefacts si leurs entrées n’ont pas changé ; invalider les dépendances lorsqu’un script ou une scène change.
- Classer les erreurs : réseau, quota, authentification, validation, rendu, droits ou intervention humaine.
- Conserver un manifeste indiquant version du code, modèles, paramètres, dates et hashes.

6. Workflow complet

1 — Idée

L’utilisateur soumet un sujet via Telegram ou demande des idées. Détection de doublons et création du projet.

2 — Cadrage

Définir audience, langue, angle, durée, format, difficulté et promesse éditoriale.

3 — Recherche

Collecter des sources primaires/fiables et enregistrer URL, titre, éditeur, date et notes utiles.

4 — Plan & script

Proposer hooks, plan, narration, exemples, limites et conclusion ; distinguer faits et hypothèses.

5 — Fact-check

Relier chaque affirmation vérifiable à une source et signaler les claims non confirmés.

6 — Validation script

Envoyer le script dans Telegram : approuver, réviser, rejeter ou commenter.

7 — Storyboard

Pour chaque scène : durée, narration, texte écran, type de visuel, caméra, animation et ratio.

8 — Assets

Générer ou rendre les visuels ; enregistrer licence, provenance, prompt et modèle.

9 — Voix off

Générer en segments, vérifier prononciation, silences, volume et durée.

10 — Sous-titres

Créer SRT/VTT, synchroniser et contrôler ponctuation, noms propres et retours à la ligne.

11 — Montage

Assembler scènes, audio, transitions, textes et musique autorisée ; exporter 16:9 et 9:16.

12 — QA

Vérifier résolution, durée, audio, noirs, sous-titres, sources, droits et divulgation IA.

13 — Revue humaine

Envoyer aperçu + rapport QA. Aucun upload public sans approbation explicite.

14 — Packaging

Proposer titres, description, chapitres, sources et miniature sans clickbait trompeur.

15 — Publication

Upload privé par défaut ; programmation publique seulement après confirmation.

16 — Suivi

Rappels, erreurs d’upload, état de publication et rapports à 24 h / 7 jours.

6.1 Profils d’export

Sortie	Paramètres par défaut	Remarques
Longue	1920×1080, 16:9, 25/30 fps, MP4 H.264 + AAC	Durée éditoriale indicative 5–10 min, à adapter au sujet.
Short	1080×1920, 9:16, 30 fps, MP4 H.264 + AAC	30–60 s comme point de départ, accroche autonome.
Audio	48 kHz AAC, niveau cohérent, sans saturation	Mesurer et écouter le résultat.
Sous-titres	SRT + VTT + incrustation facultative	Conserver une version modifiable.
Miniature	Source éditable + export 16:9	Contraste, texte court, promesse honnête.

Ne pas simplement recadrer toutes les vidéos longues : le système doit pouvoir générer un storyboard vertical ou un mini-script autonome. Le cadrage vertical doit respecter les zones de sécurité et éviter de cacher les éléments explicatifs.

7. Exigences détaillées

7.1 Gestion éditoriale

- Créer, dupliquer, archiver et annuler un projet ; conserver les versions de script.
- Champs obligatoires : sujet, angle, langue, audience, format, durée cible, statut et risque éditorial.
- Produire au moins 3 angles et 3 hooks candidats, puis laisser l’utilisateur choisir ou modifier.
- Maintenir un registre de claims : identifiant, formulation, source, état, confiance et commentaire.
- Bloquer l’approbation si une affirmation centrale reste non vérifiée ou si les droits sont inconnus.

7.2 Génération visuelle

- Types d’assets : SVG, graphique, illustration, animation Manim, scène Blender, Three.js rendu, image générée ou média sous licence.
- Chaque scène définit narration, durée, composition, mouvement, texte écran, caméra, ratio et assets requis.

- Le code de scène généré par IA s’exécute dans un worker isolé ; le rendu doit être testé avant le montage final.
- Prévisualiser chaque scène individuellement pour réduire le temps de rendu perdu.
- Prévoir un fallback déterministe (SVG, graphique ou texte animé) si un moteur échoue.
- Indiquer quand un schéma est simplifié, simulé ou non à l’échelle.

7.3 Voix off et musique

- Paramètres : fournisseur, modèle, voix, langue, accent, style, débit, prononciation et segmentation.
- Ne pas couper une phrase entre deux segments ; conserver les pauses et une marge de durée.
- Vérifier fichier audio, durée, silences anormaux, volume et clipping.
- Musique désactivée par défaut ou issue d’une bibliothèque documentée ; jamais une piste trouvée au hasard sur Internet.
- Ne pas imiter une voix identifiable sans autorisation documentée.

7.4 Sous-titres et montage

- Exporter SRT/VTT et une version incrustée facultative.
- Contrôler synchronisation, vitesse de lecture, longueur des lignes, ponctuation et zones de sécurité.
- Effets et transitions configurables par type de vidéo ; aucun effet « viral » automatique sans cohérence éditoriale.
- Utiliser les animations pour la compréhension : trajectoires, labels, zooms explicatifs, comparaisons et mises en évidence.
- Permettre de reconstruire le montage après correction d’une scène sans refaire toute la recherche.

7.5 Packaging et publication

- Générer trois titres candidats : informatif, question et curiosité honnête ; bannir les promesses non soutenues.
- Description avec résumé, sources, notes de simulation/illustration, chapitres si utiles et divulgation si requise.
- Enregistrer la miniature source et l’export final, avec variantes éventuelles.
- Upload privé par défaut ; programmation publique après confirmation explicite.
- Contrôler OAuth, permissions, quotas et paramètres de visibilité avant tout upload.

8. Bot Telegram : commandes

8.1 Sécurité

Le bot est privé et limité à une allowlist d’identifiants Telegram. Vérifier les droits à chaque commande et callback. Les actions destructrices, l’approbation finale et la publication exigent une confirmation explicite. Les boutons expirent et restent liés au projet concerné.

8.2 Catalogue de commandes

Commande	Comportement
/start	Présentation, contrôle d'accès et commandes principales.
/help	Aide classée par catégorie.
/status	État des workers, file, stockage, quotas connus et incidents.
/new [sujet]	Créer un projet ou ouvrir un assistant de création.
/ideas [thème]	Proposer des sujets avec angle, audience et risque éditorial.
/queue	Lister tâches en attente, en cours, bloquées et en erreur.
/videos	Lister les contenus par statut, langue, chaîne, format et date.
/remaining	Afficher les étapes restantes et les validations manquantes.
/open ID	Fiche du projet, statut, sorties et prochaines actions.
/preview ID	Envoyer un aperçu ou un lien local sécurisé.
/script ID	Afficher le script courant et sa version.
/sources ID	Sources, claims non vérifiés et droits à confirmer.
/approve ID	Approuver l'étape courante après résumé des risques.
/revise ID [note]	Renvoyer en révision avec une consigne.
/reject ID [raison]	Rejeter/archiver avec raison obligatoire.
/retry ID	Reprendre depuis le dernier artefact valide.
/cancel ID	Annuler une tâche, avec confirmation si rendu actif.
/schedule	Calendrier éditorial et publications programmées.
/besttime	Créneaux estimés, source des données et niveau de confiance.
/reminders	Configurer rappels de préparation, validation et publication.
/publish ID	Ouvrir une confirmation de publication, jamais publier silencieusement.
/analytics [période]	Métriques disponibles et comparaisons par format/thème.
/report [période]	Bilan quotidien, hebdomadaire ou mensuel.
/settings	Langue, chaînes, formats, durée, TTS et politique de publication.
/pause	Suspendre les nouveaux jobs sans supprimer les tâches existantes.
/resume	Reprendre la file de production.
/logs ID	Afficher erreurs nettoyées sans secrets.
/backup	Déclencher ou vérifier une sauvegarde si configurée.
/privacy	Expliquer données enregistrées et options d'export/suppression.

8.3 Boutons interactifs

- Script : Approuver / Réviser / Voir sources / Rejeter.
- Rendu : Voir aperçu / Refaire cette scène / Relancer QA / Corriger.
- Publication : Publier en privé / Programmer / Annuler ; afficher chaîne, titre, date, fuseau et visibilité.
- Erreur : Réessayer / Voir détails / Mettre en pause.
- Calendrier : +1 jour / +1 semaine / Changer créneau / Marquer publié.

8.4 Notifications automatiques

- Projet créé ; recherche terminée ; script prêt ; claim non vérifié.
- Voix terminée ; rendu en cours ; rendu terminé ; QA réussi/échoué.
- Vidéo prête à valider avec durée, formats, miniature, rapport QA et chemins.

- Publication programmée, upload échoué, publication confirmée et rappel d’analytics.
- Rappels configurables à J-1 et H-2 ; rapports quotidiens/hebdomadaires avec contenu créé, restant et erreurs.

8.5 Heures de publication

Utiliser en priorité les données propres à la chaîne, notamment le rapport YouTube Studio « Quand vos spectateurs sont sur YouTube » lorsqu’il est disponible. Sans données, proposer des créneaux comme hypothèses de test, jamais comme vérités universelles. Séparer chaînes française et anglaise, respecter le fuseau horaire et comparer plusieurs semaines.

Chaque suggestion affiche : créneau, fuseau, source, taille d’échantillon, niveau de confiance et date de réévaluation. Le bot ne doit jamais inventer des données d’audience.

9. Données et structure du dépôt

Entité	Champs essentiels
Channel	id, nom, langue, fuseau, identifiant YouTube, statut OAuth, règles éditoriales.
Project	id, sujet, angle, audience, langue, format, durée, statut, dates.
Source	id, URL, titre, éditeur, dates, type, notes, licence.
Claim	id, projet, affirmation, sources, état de vérification, confiance, commentaire.
ScriptVersion	id, projet, numéro, texte, hooks, plan, langue, approbation.
Scene	id, ordre, durée, narration, description, texte écran, caméra, ratio, état.
Asset	id, type, chemin, source, licence, prompt, modèle, hash, droits vérifiés.
AudioTrack	fournisseur, modèle, voix, langue, texte exact, chemin, durée, statut QA.
RenderJob	id, projet, état, progression, tentative, worker, sortie, erreur normalisée.
VideoExport	ratio, résolution, codec, durée, chemin, hash, rapport QA.
Approval	projet, étape, utilisateur, décision, commentaire, date.
Publication	export, chaîne, visibilité, date programmée, URL, ID YouTube, état.
AnalyticsSnapshot	vidéo, date, vues, watch time, CTR/impressions si disponibles, abonnés, revenus/RPM si disponibles.
AuditEvent	acteur, action, objet, date, résultat et métadonnées nettoyées.

9.1 Arborescence d’un projet

```
projects/{project_id}/ brief.json research/sources.json research/claims.json scripts/fr/v001.md
storyboard/storyboard.json assets/manifest.json audio/voiceover.wav subtitles/fr.srt
scenes/landscape/scene_001.mp4 scenes/vertical/scene_001.mp4 exports/landscape.mp4 exports/vertical.mp4
thumbnails/thumbnail.png qa/report.json publication/metadata.json logs/pipeline.jsonl
```

9.2 Conservation

- Conserver les versions approuvées, les sources et les manifestes de droits.
- Supprimer les fichiers temporaires selon une politique configurable, sans effacer les livrables validés.
- Ne pas stocker les clés API dans la base, les prompts publics ou les logs.
- Permettre l’export d’un projet avec scripts, sources, manifestes, sous-titres et rendus.

10. Qualité, sécurité et observabilité

10.1 Contrôles automatiques bloquants

- Script : sources présentes, claims centraux vérifiés, citations et chiffres signalés.
- Assets : droits connus ou création originale ; manifeste complet.
- Audio : fichier lisible, durée attendue, pas de silence anormal ou clipping détecté.
- Vidéo : résolution, ratio, durée, codec, fichier non vide, absence de noir ou d’image figée au-delà du seuil.
- Sous-titres : SRT/VTT valides, timestamps cohérents, pas de chevauchement interdit, zones de sécurité respectées.

- Packaging : titre non trompeur, sources incluses, mention de simulation et divulgation si nécessaire.
- Publication : utilisateur autorisé, chaîne et visibilité vérifiées, confirmation explicite.

10.2 Contrôles humains obligatoires

- Exactitude, analogies, limites et incertitudes scientifiques.
- Prononciation des termes techniques, fluidité, rythme et transitions.
- Lisibilité mobile, cohérence des effets et compréhension du montage.
- Droits des éléments tiers, divulgation IA, titre, miniature, description et programmation.

10.3 Sécurité

- Secrets via variables d'environnement ou gestionnaire de secrets ; fichiers de secrets exclus de Git.
- Exécuter les scripts générés par IA dans un worker isolé, sans secrets et avec accès système limité.
- Valider les chemins, limiter CPU/RAM/durée/taille et imposer des timeouts.
- Ne jamais afficher clés, jetons OAuth, e-mails privés ou chemins sensibles dans Telegram.
- Protéger les callbacks contre le rejeu ; auditer les actions sensibles.
- Autoriser uniquement des identifiants Telegram explicitement listés.

10.4 Observabilité

- Logs structurés : project_id, job_id, étape, durée, fournisseur, statut et code d'erreur.
- Métriques : durée par étape, taux d'échec, retries, quota, espace disque et temps de rendu.
- Alertes Telegram en cas de blocage, quota atteint, stockage insuffisant ou authentification expirée.
- Erreurs explicites avec procédure de reprise ; ne pas masquer silencieusement les erreurs.

11. Tests et critères d'acceptation

11.1 Tests attendus

- Unitaires : transitions d'état, permissions, validation sources, durée, chemins de fichiers.
- Intégration : fournisseurs, stockage, queue, bot Telegram et publication en mode test.
- End-to-end : une idée devient un export horizontal et vertical avec voix, sous-titres, sources et QA.
- Résilience : coupure réseau, quota TTS, rendu échoué, audio corrompu et OAuth expiré.
- Sécurité : utilisateur non autorisé, callback périmé, chemin malveillant, secret dans les logs.
- Régression visuelle : exports de référence pour comparer cadrage et sous-titres.

11.2 Critères d’acceptation du MVP

ID	Critère
AC-01	Une commande Telegram crée un projet avec identifiant unique.
AC-02	Sources, claims, scripts et versions sont conservés sans écrasement.
AC-03	Un claim central non vérifié bloque l’approbation du script.
AC-04	Le fournisseur TTS se remplace par configuration.
AC-05	Le storyboard produit un export 16:9 et un export 9:16.
AC-06	SRT/VTT sont exportés et passent les contrôles.
AC-07	Telegram reçoit l’aperçu, les sources et le rapport QA.
AC-08	Aucun upload public sans validation d’un utilisateur autorisé.
AC-09	Un job échoué reprend depuis le dernier artefact valide.
AC-10	Aucun secret dans Git, logs ou messages du bot.
AC-11	/videos distingue terminé, en cours, bloqué, échoué et publié.
AC-12	/remaining indique clairement les tâches et validations restantes.
AC-13	/besttime montre la source et n’invente pas les données d’audience.
AC-14	Chaque export possède manifeste d’assets et rapport QA.
AC-15	Une vidéo pilote est relue manuellement avant la production.

12. Roadmap de réalisation

Phase	Livrables	Critère de sortie
0 — Cadrage	Charte éditoriale, exemple de vidéo, règles de sources et approbations.	Spécification confirmée.
1 — Socle	Repo, base, queue, statuts, logs, stockage, permissions Telegram.	Bot suit un projet simulé.
2 — Recherche/script	Sources, registre de claims, prompts, versions, validation Telegram.	Script sourcé approuvable.
3 — TTS/sous-titres	Gemini TTS, fallback, segmentation, normalisation, SRT/VTT.	Audio relu et sous-titres synchronisés.
4 — Rendu	Storyboard JSON, SVG/Manim, Blender optionnel, FFmpeg.	Long + Short de test.
5 — QA/packaging	Rapports, miniature, métadonnées, droits, divulgation IA.	Dossier complet prêt à valider.
6 — Publication	OAuth, upload privé, programmation après validation.	Publication test contrôlée.
7 — Analytics	Import métriques et rapports Telegram.	Boucle d’amélioration basée sur les données.

Ordre conseillé : ne pas commencer par l’upload automatique ou une génération 3D complexe. Garantir d’abord une chaîne fiable de l’idée au fichier exporté, avec sources et validation.

13. Déploiement et configuration

13.1 Variables de configuration indicatives

```
APP_ENV=development DATABASE_URL=sqlite:///./data/app.db TELEGRAM_BOT_TOKEN=...
TELEGRAM_ALLOWED_USER_IDS=123456789 GEMINI_API_KEY=... TTS_PROVIDER=gemini TTS_MODEL= TTS_FALLBACK=piper
YOUTUBE_CLIENT_SECRET_FILE=./secrets/youtube-client-secret.json YOUTUBE_DEFAULT_VISIBILITY=private
PUBLICATION_REQUIRES_APPROVAL=true DEFAULT_LANGUAGE=fr DEFAULT_TIMEZONE=Europe/Paris LANDSCAPE_WIDTH=1920
LANDSCAPE_HEIGHT=1080 VERTICAL_WIDTH=1080 VERTICAL_HEIGHT=1920 MAX_RENDER_WORKERS=1 MAX_RETRIES=3
```

Les valeurs réelles ne doivent jamais être copiées dans un document public ou versionnées. Les noms ci-dessus sont des exemples, pas des clés.

13.2 Déploiement initial

- Développement local Windows/Linux avec environnement Python isolé.

- Installer FFmpeg puis tester Manim et Blender séparément ; configurer les chemins exécutables.
- Commencer avec un worker de rendu pour éviter de saturer la machine.
- Backuper base, scripts, sources et manifestes ; appliquer une rétention configurable aux rendus temporaires.
- Prévoir un mode dry-run pour tester les commandes de publication sans publication réelle.
- Surveiller les quotas, les conditions des API et l'évolution des offres gratuites.

13.3 Structure de dépôt indicative

```
youtube-studio/ apps/api/ # FastAPI facultatif apps/telegram_bot/ apps/dashboard/ # optionnel
services/research/ services/editorial/ services/factcheck/ services/storyboard/ services/assets/
services/tts/ services/subtitles/ services/rendering/ services/quality/ services/youtube/
services/analytics/ providers/llm/ tts/ image/ renderer/ youtube/ workers/ migrations/ tests/ configs/
docs/ scripts/ data/ # exclu de Git secrets/ # exclu de Git .env.example pyproject.toml README.md
```

14. Risques et décisions

Risque	Mesure de réduction
Refus de monétisation	Originalité réelle, scripts/visuels distincts, narration utile, contrôle humain.
Faits erronés	Sources primaires, registre des claims, revue humaine, incertitudes visibles.
Quota TTS épuisé	Surveillance, retry limité, fallback Piper et mise en attente.
Droits incertains	Registre des licences, provenance documentée, assets originaux ou autorisés.
Rendu 3D défectueux	Prévisualisation scène par scène, logs, limites de calcul, fallback SVG/Manim.
Mauvais créneaux	Données Analytics propres à la chaîne, tests sur plusieurs semaines.
Publication accidentelle	Privé par défaut, allowlist, confirmation et audit.
Coût inattendu	Budget/quota configurable, alertes, fournisseur alternatif.
Secrets exposés	Variables d'environnement, nettoyage des logs, permissions et rotation.

Décisions recommandées pour le MVP

- Un propriétaire et une allowlist Telegram ; une chaîne au départ.
- Gemini TTS comme fournisseur principal expérimental, Piper comme fallback local.
- SVG + Manim + FFmpeg en première version ; Blender uniquement pour les scènes 3D pertinentes.
- Upload privé uniquement au début ; programmation publique après tests et validation.
- Validation humaine obligatoire à trois étapes : script, rendu final, publication.
- Mesurer les métriques réelles plutôt que se baser sur des estimations de RPM.

15. Sources officielles

Les politiques, tarifs et quotas évoluent. Relire ces références avant chaque mise en production et conserver la date de vérification dans la documentation du projet.

Politique de monétisation YouTube
https://support.google.com/youtube/answer/1311392?hl=fr
Contenu inauthentique, répétitif et réutilisé.

Divulgation des contenus IA YouTube
https://support.google.com/youtube/answer/14328491?hl=fr
Quand signaler les contenus réalistes générés ou modifiés.

Gemini TTS — documentation officielle
https://ai.google.dev/gemini-api/docs/speech-generation
Modèles, voix et génération audio contrôlable.

Tarification Gemini API
https://ai.google.dev/gemini-api/docs/pricing
Niveaux gratuits, tarifs et quotas selon les modèles.

Facturation Gemini API
https://ai.google.dev/gemini-api/docs/billing
Niveaux et conditions de facturation.

Audience YouTube — horaires
https://support.google.com/youtube/answer/9314416?hl=fr
Rapport « Quand vos spectateurs sont sur YouTube ».

Commandes de bots Telegram
https://core.telegram.org/bots/features#commands
Expérience et commandes des bots.

Telegram Bot API
https://core.telegram.org/bots/api
Interface technique officielle.

Manim Community
https://docs.manim.community/
Animations programmatiques.

Blender Python API
https://docs.blender.org/api/current/
Automatisation 3D.

FFmpeg documentation
https://ffmpeg.org/documentation.html
Encodage, assemblage et traitement média.

Étape suivante recommandée : convertir cette spécification en backlog technique (épics, tickets, schéma SQL, contrats des fournisseurs et tests) avant de coder l'ensemble du pipeline.